Open in app ↗ Sign up Sign in

Medium Search Write

Building a Modular ETL Pipeline with Python, DBT, and Supabase

Charlesisaac Follow 2 min read · Oct 11, 2025

1

In the world of data engineering, simplicity and scalability often feel like opposing forces. But what if you could build a clean, modular ETL pipeline that's both easy to manage and ready for production? That's exactly what I set out to do — and in this article, I'll walk you through how I built it using Python, DBT, Supabase, and Docker.

🧱 The Goal

I wanted to create a lightweight ETL pipeline that:

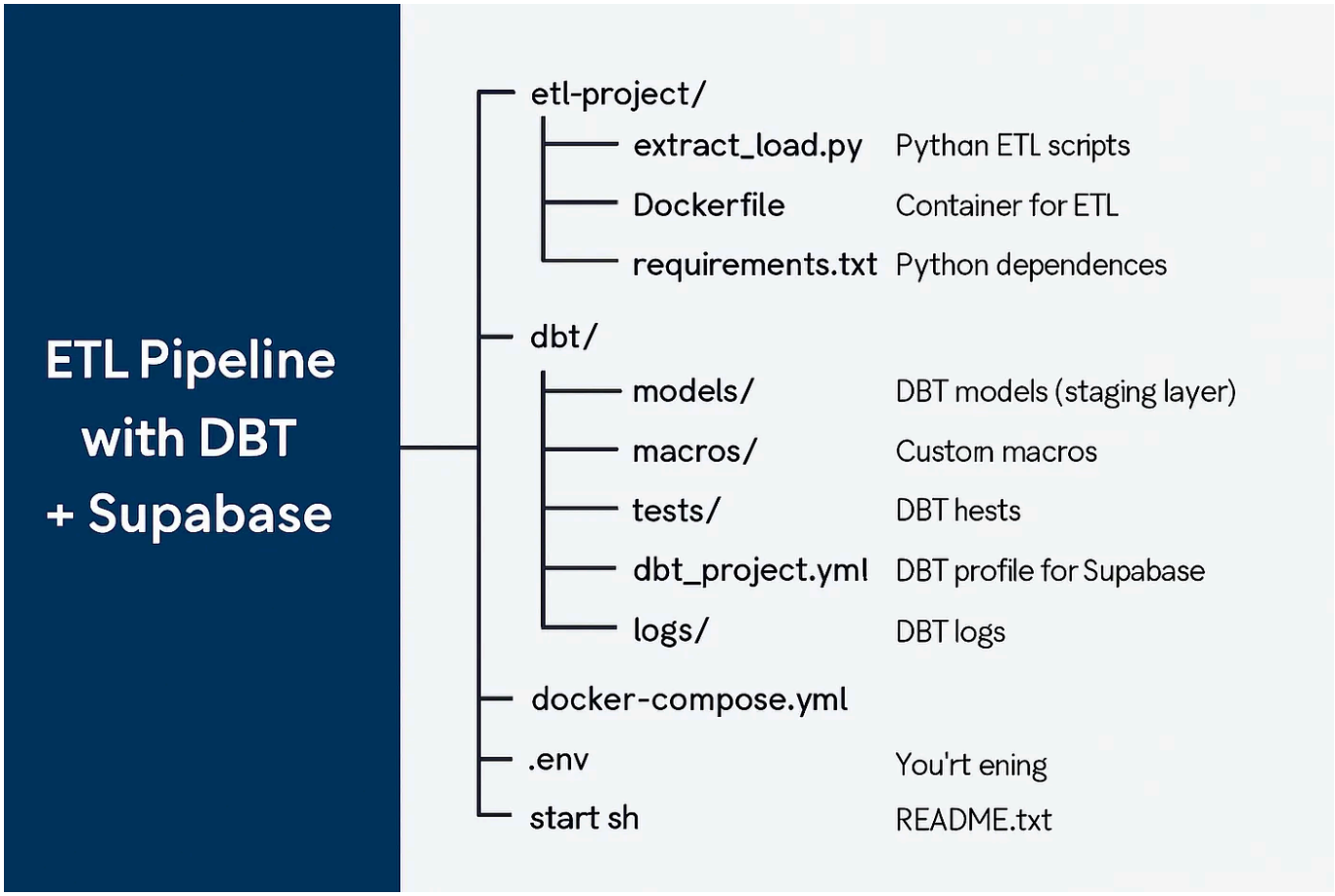
- Extracts raw data from a PostgreSQL database hosted on Supabase
- Transforms it into clean, validated staging models using DBT
- Runs inside Docker containers for portability
- Automates daily execution with cron
- Is version-controlled, documented, and ready for dashboard integration

⚙️ The Stack

- **Supabase:** PostgreSQL backend with a slick UI and easy setup
- **Python:** For writing the ETL logic
- **DBT:** For transforming and testing data
- **Docker:** To containerize the entire pipeline
- **Cron:** For scheduling daily runs

🧠 The Process

1. Designing the Structure



This modular layout keeps everything organized and scalable.

2. Writing the ETL Script

The `extract_load.py` script connects to Supabase using credentials from `.env`, creates staging tables, and copies data from raw tables. It's simple, readable, and easy to extend.

3. Modeling with DBT

Inside `dbt/dbt_project/models/`, I created SQL models like:

sql

```
-- staging_orders.sql
SELECT * FROM public.orders
```

Each model is materialized as a view or table, and tested using DBT's built-in schema tests (`not_null`, `unique`, `relationships`).

4. Containerizing with Docker

I wrote a `Dockerfile` for the ETL script and used `docker-compose.yml` to orchestrate both the ETL and DBT containers. This makes the pipeline portable and easy to deploy anywhere.

5. Automating with Cron

The `start.sh` script not only runs the pipeline manually, but also adds a cron job to execute it daily at midnight:

bash

```
0 0 * * * docker exec etl python etl/extract_load.py
```

This ensures the pipeline runs consistently without manual intervention.

✅ Final Touches

- Added a clean `README.md` with setup instructions, architecture, and usage
- Generated DBT documentation with `dbt docs generate`
- Version-controlled everything with Git
- Ready to connect to Metabase or Superset for dashboarding

🚀 What's Next

This pipeline is now ready for:

- Real-time data ingestion
- Incremental models
- CI/CD integration with GitHub Actions
- Visualization with BI tools

💬 Final Thoughts

Building this pipeline was a rewarding exercise in simplicity, modularity, and automation. Whether you're a solo data engineer or part of a growing analytics team, this setup offers a clean foundation for scalable data workflows.